LAPORAN PRATIKUM PEKAN 5

STRUKTUR DATA

disusun Oleh:

ALIF MUHAMMAD IHSAN

NIM 2511531004

Dosen Pengampu : Dr.Wahyudi S.T, M.T

Asisten : M Galid Avero

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2025

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum **Struktur Data** ini dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban atas kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai sarana untuk memperdalam pemahaman mengenai konsep dasar struktur data, implementasi algoritma, serta penerapan logika pemrograman dalam kehidupan nyata.

Dalam penyusunan laporan ini, penulis menyadari masih terdapat banyak kekurangan, baik dari segi isi maupun penyajian. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang.

Ucapan terima kasih penulis sampaikan kepada:

- Dosen pengampu mata kuliah **Struktur Data** yang telah memberikan arahan dan bimbingan.
- Asisten praktikum yang senantiasa membantu dalam memahami materi dan menyelesaikan permasalahan teknis.
- Rekan-rekan mahasiswa yang turut berdiskusi dan bekerja sama selama kegiatan praktikum berlangsung.

Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi penulis sendiri maupun bagi pembaca yang ingin mempelajari lebih lanjut mengenai struktur data.

Daftar Isi

KATA PENGANTAR i
BAB I 3
PENDAHULUAN 3
1.1 Latar Belakang 3
1.2 Tujuan 4
1.3 Manfaat Praktikum 4
BAB II 5
PEMBAHASAN 5
2.1 Kode Progaram 5

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu aspek fundamental dalam ilmu komputer yang berperan penting dalam pengelolaan dan pengorganisasian data. Dengan adanya struktur data, proses penyimpanan, pengambilan, serta manipulasi data dapat dilakukan secara lebih efisien dan terstruktur. Pemahaman mengenai struktur data menjadi dasar bagi pengembangan algoritma yang optimal, sehingga sangat berpengaruh terhadap performa suatu program maupun sistem.

Melalui praktikum struktur data, mahasiswa tidak hanya mempelajari konsep teoritis, tetapi juga mengimplementasikan langsung berbagai bentuk struktur data seperti array, linked list, stack, queue, dan tree dalam bahasa pemrograman. Praktikum ini bertujuan untuk melatih keterampilan analisis logika, pemecahan masalah, serta kemampuan debugging yang diperlukan dalam pengembangan perangkat lunak.

Selain itu, praktikum ini juga memberikan pengalaman nyata dalam menghubungkan teori dengan praktik, sehingga mahasiswa dapat memahami bagaimana suatu algoritma bekerja di balik layar dan bagaimana pemilihan struktur data yang tepat dapat memengaruhi efisiensi program. Dengan demikian, kegiatan praktikum struktur data menjadi sarana penting untuk membangun fondasi keilmuan yang kuat dalam bidang informatika.

1.2 Tujuan

1. Memahami konsep dasar struktur data dan penerapannya dalam pemrograman.
2. Melatih keterampilan dalam mengimplementasikan berbagai jenis struktur data seperti array, linked list, stack, queue, dan tree menggunakan bahasa pemrograman.
3. Mengembangkan kemampuan analisis logika dan pemecahan masalah melalui studi kasus yang diberikan dalam praktikum.
4. Melatih keterampilan debugging serta troubleshooting program agar dapat menemukan dan memperbaiki kesalahan secara sistematis.

1.3 Manfaat Praktikum

1. Memberikan pengalaman langsung dalam mengimplementasikan struktur data sehingga mahasiswa lebih mudah dalam memahami konsep yang abstrak
2. Meningkatkan keterampilan pemograman, khususnya dalam efesiensi algoritma dan pengelolaan data.
3. Menjadi bekal penting untuk mata kuliah lanjutan maupun proyek pengembangan perangkat lunak di masa depan.
4. Membantu mahasiswa memahami pentingnya pemilihan struktur data yang tepat dalam meningkatkan peforma program.

BAB II

PEMBAHASAN

2.1 Kode Progaram

a. HapusSLL_2511531004

```
package Pekan5_2511531004;

public class hapusSLL_2511531004 {
// fungsi untuk menghapus head
    public static NodeSLL_2511531004 deleteHead (NodeSLL_2511531004
head_1004) {
            // jika sll kosong
            if (head_1004 == null )
                    return null;

            // pindahkan head ke node berikutnya
            head_1004 = head_1004.next_1004;

            // return head baru
            return head_1004;
    }

    public static NodeSLL_2511531004 removeLastNode
(NodeSLL_2511531004 head_1004) {
            if(head_1004 == null ) {
                    return null;
            }

            // jika list satu node, hapus node dan return null
            if ( head_1004.next_1004 == null) {
                    return null;
            }
            // temukan node terakhir kedua
            NodeSLL_2511531004 secondLast = head_1004;

            while (secondLast.next_1004.next_1004 != null ) {
                    secondLast = secondLast.next_1004;
            }
            // hapus nnode terakhir
            secondLast.next_1004 = null;

            return head_1004;
    }

    // fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511531004 deleteNode(NodeSLL_2511531004
head_1004, int position_1004) {
            NodeSLL_2511531004 temp_1004 = head_1004;
            NodeSLL_2511531004 prev_1004 = null;
            // jika linked list null
            if(temp_1004 == null)
                    return head_1004;
```

```
        // kasus 1 : head dihapus
        if (position_1004 == 1 ) {
                head_1004 = temp_1004.next_1004;
                return head_1004;
        }
                // kasus 2 menghapous node di tengah
                // telusuri node yang dihapus

                for (int i = 1; temp_1004 != null && i <
position_1004; i++ ) {
                        prev_1004 = temp_1004;
                        temp_1004 = temp_1004.next_1004;
                }

                if ( temp_1004 != null ) {
                        prev_1004.next_1004 = temp_1004.next_1004;
                }
                else {
                        System.out.println("data tidak ada ");
                }
                return head_1004;
    }


                // fungsi mencetak sll
                public static void printList_2511531004
(NodeSLL_2511531004 head_1004) {
                        NodeSLL_2511531004 curr = head_1004;
                        while ( curr.next_1004 != null) {
                                System.out.print(curr.data_1004 + " ---
-> ");
                                curr = curr.next_1004;
                        }
                        if (curr.next_1004 == null) {
                                System.out.print(curr.data_1004);
                        }
                        System.out.println();
                }

                public static void main (String[] args ) {

                        NodeSLL_2511531004 head = new
NodeSLL_2511531004 (1);
                        head.next_1004 = new NodeSLL_2511531004 (2);
                        head.next_1004.next_1004 = new
NodeSLL_2511531004 (3);
                        head.next_1004.next_1004.next_1004 = new
NodeSLL_2511531004(4);
                        head.next_1004.next_1004.next_1004.next_1004
= new NodeSLL_2511531004(5);

    head.next_1004.next_1004.next_1004.next_1004.next_1004 = new
NodeSLL_2511531004(6);

                        // cetak list awal
                        System.out.print("list awal : ");
```

```
                        printList_2511531004 (head);

                        // hapus head
                        head = deleteHead (head) ;
                        System.out.print("list setelah head di hapus
: ");
                        printList_2511531004 (head);

                        // hapus node terakhir
                        head = removeLastNode (head);
                        System.out.print("list setelah simpul
terakhir di hapus : ");
                        printList_2511531004 (head);

                        // deleting node at position 2
                        int position = 2;
                        head = deleteNode (head, position);

                        // print list after deletd

                        System.out.print("list setelah posisi 2 di
hapus : ");
                        printList_2511531004 (head);


                }


    }
```

b. NodeSLL_2511531004

```
package Pekan5_2511531004;

public class NodeSLL_2511531004 {
    // node bagian data
    int data_1004;
    // pointer ke node berikutnya
    NodeSLL_2511531004 next_1004;

    // Konstruktor menginisiasi node dengan data

    public NodeSLL_2511531004 (int data_1004)
    {

            this.data_1004 = data_1004;
            this.next_1004 = null;
    }
}
```

c. PencarianSLL_2511531004

```
package Pekan5_2511531004;

public class PencarianSLL_2511531004 {
	static boolean searchKey_2511531004 ( NodeSLL_2511531004
head_1004, int key_1004 ) {
		NodeSLL_2511531004 curr = head_1004;
		while ( curr != null ) {
			if(curr.data_1004 == key_1004)
				return true;
			curr = curr.next_1004;
		}
		return false;
	}

		public static void traversal_2511531004 (NodeSLL_2511531004
head_1004) {
			// mulai dari head
			NodeSLL_2511531004 curr = head_1004;
			// telusuri sampai pointer null
			while (curr != null) {
				System.out.print
				(" " + curr.data_1004 );
				curr = curr.next_1004;
			}
			System.out.println();
		}
		public static void main (String [] args) {
			NodeSLL_2511531004 head = new NodeSLL_2511531004
(14);
			head.next_1004 = new NodeSLL_2511531004 (21);
			head.next_1004.next_1004 = new NodeSLL_2511531004
(13);
			head.next_1004.next_1004.next_1004 = new
NodeSLL_2511531004 (30);
			System.out.print("Penulusuran SLL : ");
			traversal_2511531004 (head);
			// data yang akan dicari
			int key_1004 = 30;
			System.out.println("cari data " + key_1004 + " = ");
			if (searchKey_2511531004 (head, key_1004) )
				System.out.print("ketemu");
				else
					System.out.println("tidak ada");


		}
}
```

d. TambahSLL_2511531004

```
package Pekan5_2511531004;

public class tambahSLL_2511531004 {

	public static NodeSLL_2511531004
insertAtFront_2511531004(NodeSLL_2511531004 head_1004, int value_1004) {
		NodeSLL_2511531004 new_node = new NodeSLL_2511531004
(value_1004);
		new_node.next_1004 = head_1004;
		return new_node;

	}
	// fungsi menambahkan node diakhir sll
	public static NodeSLL_2511531004 insertAtEnd_2511531004
(NodeSLL_2511531004 head_1004, int value_1004) {
		// buat sebuah node dengan sebuah nilai
		NodeSLL_2511531004 newNode = new NodeSLL_2511531004
(value_1004) ;
		// jika list kosong maka node jadi head
		if (head_1004 == null ) {
			return newNode;
		}

		// simpan head ke variabel sementara
		NodeSLL_2511531004 last = head_1004;
		// telurusi node akhir
		while(last.next_1004 != null) {
			last = last.next_1004;
		}
		// ubah pointer
		last.next_1004 = newNode;
		return head_1004;


		}

	static NodeSLL_2511531004 GetNode ( int data_1004 ) {
		return new NodeSLL_2511531004(data_1004);

	}

	static NodeSLL_2511531004 insertPos_2511531004(NodeSLL_2511531004
headNode_1004, int position_1004, int value_1004) {
		NodeSLL_2511531004 head = headNode_1004;
		if(position_1004 < 1) {
			System.out.println("Invalid Position");
			if (position_1004 == 1 ) {
				NodeSLL_2511531004 new_node = new
NodeSLL_2511531004 (value_1004);
				new_node.next_1004 = head;
				return new_node;
			} else {
				while (position_1004-- != 0) {
					if(position_1004 == 1 ) {
```

```
                                        NodeSLL_2511531004 newNode =
GetNode(value_1004);
                                        newNode.next_1004 =
headNode_1004.next_1004;
                                        headNode_1004.next_1004 =
newNode;
                                        break;
                                }
                                headNode_1004 =
headNode_1004.next_1004;
                        }
                        if (position_1004 != 1 ) {
                        System.out.println("Posisi di luar
jangkauan");
                        }

                        }
                }
            return head;
            }


                public static void printList_2511531004
(NodeSLL_2511531004 head_1004) {
                        NodeSLL_2511531004 curr = head_1004;

                        while (curr.next_1004  != null ) {
                                System.out.print(curr.data_1004 + " ---
> " );
                                curr = curr.next_1004;
                        }
                        if ( curr.next_1004 == null ) {
                                System.out.print(curr.data_1004);
                                System.out.println();
                        }
                }


            public static void main(String [] args) {
                // buat linked list
                NodeSLL_2511531004 head = new NodeSLL_2511531004
(2);
                head.next_1004 = new NodeSLL_2511531004 (3);
                head.next_1004.next_1004 = new NodeSLL_2511531004
(5);
                head.next_1004.next_1004.next_1004 = new
NodeSLL_2511531004 (6);

                // cetak list asli
                System.out.println("Senarai berawal dari : " );
                printList_2511531004(head);

                // tambahkan node baru di depan

                System.out.println("tambah 1 simpul di bagian depan
: ");
                int data_1004 = 1;
```

```
                    head = insertAtFront_2511531004 (head, data_1004);

                    // cetak update list
                    printList_2511531004 (head);
                    System.out.println("tambah 1 simpul di belakang :
");
                    int data2_1004 = 7;
                    head = insertAtEnd_2511531004(head, data2_1004);

                    // cetak update list
                    printList_2511531004(head);
                    System.out.println("tambah 1 simpul ke data 4 : ");
                    int data3_1004 = 4;
                    int pos_1004 = 4;
                    head = insertPos_2511531004(head, pos_1004,
data3_1004);

                    // cetak update list
                    printList_2511531004 (head);

            }
}
```

2.2 Langkah Kerja

a) HapusSLL_2511531004

Membuat Linked List

- Node dibuat manual: 1 → 2 → 3 → 4 → 5 → 6.
- head menunjuk ke node pertama (nilai 1).

Mencetak List Awal

- Fungsi printList_2511531004 menelusuri node dari head sampai akhir.
- Hasil cetakan: 1 ----> 2 ----> 3 ----> 4 ----> 5 ----> 6.

Menghapus Head

- Fungsi deleteHead memindahkan head ke node berikutnya.
- Node pertama (nilai 1) terhapus.
- List menjadi: 2 → 3 → 4 → 5 → 6.
- Cetakan: 2 ----> 3 ----> 4 ----> 5 ----> 6.

Menghapus Node Terakhir

- Fungsi removeLastNode mencari node kedua terakhir.
- Node terakhir (nilai 6) diputuskan.
- List menjadi: 2 → 3 → 4 → 5.
- Cetakan: 2 ----> 3 ----> 4 ----> 5.

Menghapus Node di Posisi Tertentu (posisi = 2)

- Fungsi deleteNode menelusuri sampai posisi ke-2.
- Node dengan nilai 3 dihapus.
- List menjadi: 2 → 4 → 5.
- Cetakan: 2 ----> 4 ----> 5.

b) NodeSLL_2511531004

Deklarasi Class

- public class NodeSLL_2511531004 Membuat sebuah class bernama NodeSLL_2511531004 yang berfungsi sebagai **struktur node** untuk Single Linked List (SLL).

Deklarasi Atribut

- int data_1004; → Menyimpan nilai/data dari node.
- NodeSLL_2511531004 next_1004; → Merupakan pointer/referensi ke node berikutnya dalam linked list. Dengan dua atribut ini, setiap node bisa menyimpan data sekaligus menghubungkan diri ke node lain.

Konstruktor

- public NodeSLL_2511531004(int data_1004) Konstruktor dipanggil saat membuat objek node baru.
- this.data_1004 = data_1004; → Mengisi data node dengan nilai yang diberikan.
- this.next_1004 = null; → Secara default, node baru tidak terhubung ke node lain (pointer next bernilai null).

c) PencarianSLL_2511531004

Membuat Linked List

- Program membuat node secara manual: 14 → 21 → 13 → 30
- head menunjuk ke node pertama (nilai 14).

Traversal (Penelusuran List)

- Fungsi traversal_2511531004 menelusuri node dari head hingga null.
- Setiap data dicetak berurutan.
- Hasil cetakan:

  Penulusuran SLL : 14 21 13 30

Menentukan Data yang Dicari

- Program menetapkan key_1004 = 30.
- Artinya, program akan mencari apakah angka 30 ada di dalam linked list.

Proses Pencarian (Search)

- Fungsi searchKey_2511531004 bekerja dengan cara:
  - Mulai dari head.
  - Bandingkan curr.data_1004 dengan key_1004.
  - Jika sama → return true.
  - Jika tidak sama → lanjut ke node berikutnya.
  - Jika sampai akhir tidak ditemukan → return false.

Hasil Pencarian

- Karena data 30 ada di node terakhir, fungsi mengembalikan true.
- Output:

  cari data 30 =

  ketemu

d). tambahSLL_2511531004

1. **Membuat Linked List Awal**
    - Node dibuat manual: 2 → 3 → 5 → 6
    - head menunjuk ke node pertama (nilai 2).
    - Cetakan awal:

Senarai berawal dari :

2 ---> 3 ---> 5 ---> 6

2. **Menambahkan Node di Depan (Insert at Front)**
    - Fungsi insertAtFront_2511531004 membuat node baru dengan nilai 1.
    - Node baru menunjuk ke head lama.
    - head diperbarui ke node baru.
    - List menjadi: 1 → 2 → 3 → 5 → 6

3. Menambahkan Node di Belakang (Insert at End)

- Fungsi insertAtEnd_2511531004 membuat node baru dengan nilai 7.
- Program menelusuri sampai node terakhir (6).
- Node terakhir menunjuk ke node baru (7).
- List menjadi: 1 → 2 → 3 → 5 → 6 → 7

4. Menambahkan Node di Posisi Tertentu (Insert at Position)

- Fungsi insertPos_2511531004 menambahkan node baru dengan nilai 4 di posisi ke-4.
- Program menelusuri sampai node ke-3 (3).
- Node baru (4) disisipkan setelah node 3.
- List menjadi: 1 → 2 → 3 → 4 → 5 → 6 → 7

BAB III

PENUTUP

3.1 Kesimpulan

Pemahaman Dasar Node dan Linked List

- Class NodeSLL_2511531004 berhasil mendefinisikan struktur node dengan data dan pointer next.
- Ini menjadi pondasi untuk membangun **Single Linked List (SLL)**.

Operasi Penambahan Node

- Program tambahSLL_2511531004 menunjukkan cara menambahkan node di depan, di belakang, dan di posisi tertentu.
- Hasilnya, list dapat berkembang dinamis sesuai kebutuhan.

Operasi Penghapusan Node

- Program hapusSLL_2511531004 mengimplementasikan penghapusan node di head, di akhir, dan di posisi tertentu.
- Hal ini melatih logika traversal dan manipulasi pointer agar list tetap konsisten.

Operasi Pencarian Node

- Program PencarianSLL_2511531004 menelusuri list untuk mencari data tertentu.
- Jika data ditemukan, program mengembalikan hasil "ketemu", jika tidak maka "tidak ada".

Alur Belajar Terstruktur

- Dari membuat node → menambah node → menghapus node → mencari node.
- Ini menunjukkan progres yang sistematis dalam memahami **struktur data linked list**.

3.2 Saran

Validasi Input

- Tambahkan pengecekan agar posisi yang dimasukkan tidak di luar jangkauan.
- Pastikan data yang dicari atau ditambahkan tidak kosong.

Pemformatan Output

- Gunakan format tabel atau tampilan lebih rapi agar hasil traversal mudah dibaca.
- Misalnya: [2] → [3] → [5].

Fitur Tambahan

- **Update Node**: ubah data pada posisi tertentu.
- **Hitung Panjang List**: tampilkan jumlah node.
- **Reverse List**: membalik urutan linked list.
- **Simpan ke File**: agar data tidak hilang saat program ditutup.

Struktur Kode Modular

- Pisahkan fungsi ke dalam class layanan (misalnya LinkedListService) agar lebih terorganisir.
- Hal ini memudahkan perawatan dan pengembangan program.

DAFTAR PUSTAKA

Oracle. (2024). *Java™ Platform, Standard Edition Documentation*. Oracle. https://docs.oracle.com/javase/8/docs/api/

Deitel, P. J., & Deitel, H. M. (2017). *Java: How to Program (Early Objects)* (11th ed.). Pearson Education.

Liang, Y. D. (2019). *Introduction to Java Programming and Data Structures* (11th ed.). Pearson.

Wirth, N. (2006). *Algorithms and Data Structures*. Springer.

GeeksforGeeks. (2025). *Java Linked List – Insertion, Deletion, and Search Operations*. https://www.geeksforgeeks.org/data-structures/linked-list/

TutorialsPoint. (2025). *Java – Linked List Implementation*.

Tugas Struktur Data Pekan 5

a. Kode Program Pasien_2511531004

```
package Pekan5_2511531004;

public class Pasien_2511531004 {
    private String namaPasien_1004;
    private String penyakit_1004;
    private int nomorAntrian_1004;
    private Pasien_2511531004 next_1004;

    // Constructor
    public Pasien_2511531004(String namaPasien_1004, String
penyakit_1004, int nomorAntrian_1004) {
        this.namaPasien_1004 = namaPasien_1004;
        this.penyakit_1004 = penyakit_1004;
        this.nomorAntrian_1004 = nomorAntrian_1004;
        this.next_1004 = null;
    }

    // Getter & Setter

    public String getNamaPasien_1004() {
       return namaPasien_1004; }

    public void setNamaPasien_1004(String namaPasien_1004)
    { this.namaPasien_1004 = namaPasien_1004; }

    public String getPenyakit_1004() {
       return penyakit_1004; }

    public void setPenyakit_1004(String penyakit_1004)
    { this.penyakit_1004 = penyakit_1004; }

    public int getNomorAntrian_1004() {
       return nomorAntrian_1004; }

    public void setNomorAntrian_1004(int nomorAntrian_1004)
    { this.nomorAntrian_1004 = nomorAntrian_1004; }

    public Pasien_2511531004 getNext_1004() {
       return next_1004; }

    public void setNext_1004(Pasien_2511531004 next_1004)
    { this.next_1004 = next_1004; }
}
```

b. Kode program RumahSakit_2511531004

```
package Pekan5_2511531004;

import java.util.Scanner;

public class RumahSakit_2511531004 {
    private Pasien_2511531004 head_1004;
    private int counter_1004;

    public RumahSakit_2511531004() {
        head_1004 = null;
        counter_1004 = 0;
    }

    // Menambahkan pasien
    public void daftarkanPasien_1004(String nama, String penyakit) {
        counter_1004++;
        Pasien_2511531004 newPasien = new Pasien_2511531004(nama,
penyakit, counter_1004);

        if (head_1004 == null) {
            head_1004 = newPasien;
        } else {
            Pasien_2511531004 temp = head_1004;
            while (temp.getNext_1004() != null) {
                temp = temp.getNext_1004();
            }
            temp.setNext_1004(newPasien);
        }
        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian:
" + counter_1004);
    }

    // Menghapus Head
    public void panggilPasien_1004() {
        if (head_1004 == null) {
            System.out.println("Tidak ada pasien dalam antrian.");
        } else {
            System.out.println("Memanggil Pasien: " +
head_1004.getNamaPasien_1004() +
                                " | Keluhan: " +
head_1004.getPenyakit_1004() +
                                " | Nomor Antrian: " +
head_1004.getNomorAntrian_1004());
            head_1004 = head_1004.getNext_1004();
        }
    }

    // Menampilkan
    public void tampilkanAntrian_1004() {
        if (head_1004 == null) {
            System.out.println("Antrian kosong.");
        } else {
            Pasien_2511531004 temp = head_1004;
            while (temp != null) {
```

```
                System.out.println("Nomor Antrian: " +
temp.getNomorAntrian_1004() +
                                        " | Nama: " +
temp.getNamaPasien_1004() +
                                        " | Keluhan: " +
temp.getPenyakit_1004());
                temp = temp.getNext_1004();
            }
        }
    }

    // mencari pasien
    public void cariPasien_1004(String nama) {
        Pasien_2511531004 temp = head_1004;
        boolean found = false;
        while (temp != null) {
            if (temp.getNamaPasien_1004().equalsIgnoreCase(nama)) {
                System.out.println("Pasien ditemukan: " +
temp.getNamaPasien_1004() +
                                        " | Keluhan: " +
temp.getPenyakit_1004() +
                                        " | Nomor Antrian: " +
temp.getNomorAntrian_1004());
                found = true;
                break;
            }
            temp = temp.getNext_1004();
        }
        if (!found) {
            System.out.println("Pasien dengan nama '" + nama + "' tidak
ditemukan.");
        }
    }

    // Status
    public void cekStatusAntrian_1004() {
        if (head_1004 == null) {
            System.out.println("Antrian kosong.");
        } else {
            int total = 0;
            Pasien_2511531004 temp = head_1004;
            while (temp != null) {
                total++;
                temp = temp.getNext_1004();
            }
            System.out.println("Jumlah total pasien: " + total);
            System.out.println("Pasien terdepan: " +
head_1004.getNamaPasien_1004() +
                                    " | Keluhan: " +
head_1004.getPenyakit_1004());
        }
    }

    // Main Menu
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        RumahSakit_2511531004 rs = new RumahSakit_2511531004();
```

```java
        int pilihan;

        do {
            System.out.println("\n=== Antrian Rumah Sakit NIM:
2511531004 ===");
            System.out.println("1. Daftarkan Pasien (Insert)");
            System.out.println("2. Panggil Pasien (Delete Head)");
            System.out.println("3. Tampilkan Antrian (Display)");
            System.out.println("4. Cari Pasien (Search)");
            System.out.println("5. Cek Status Antrian");
            System.out.println("6. Keluar");
            System.out.print("Pilihan: ");
            pilihan = sc.nextInt();
            sc.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan Nama Pasien: ");
                    String nama = sc.nextLine();
                    System.out.print("Masukkan Keluhan: ");
                    String penyakit = sc.nextLine();
                    rs.daftarkanPasien_1004(nama, penyakit);
                    break;
                case 2:
                    rs.panggilPasien_1004();
                    break;
                case 3:
                    rs.tampilkanAntrian_1004();
                    break;
                case 4:
                    System.out.print("Masukkan Nama Pasien yang dicari:
");
                    String cari = sc.nextLine();
                    rs.cariPasien_1004(cari);
                    break;
                case 5:
                    rs.cekStatusAntrian_1004();
                    break;
                case 6:
                    System.out.println("Keluar dari program...");
                    break;
                default:
                    System.out.println("Pilihan tidak valid.");
            }
        } while (pilihan != 6);
        sc.close();
    }
}
```

Output

```
Pilihan: 1
Masukkan Nama Pasien: Alif
Masukkan Keluhan: badan panas
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511531004 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: joseph
Masukkan Keluhan: sakit perut
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== Antrian Rumah Sakit NIM: 2511531004 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Jumlah total pasien: 2
Pasien terdepan: Alif | Keluhan: badan panas

=== Antrian Rumah Sakit NIM: 2511531004 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan:
```